



UNIVERSITATEA POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Programul de studii: E-Guvernare



RAPORT DE CERCETARE 3

Sistem automatizat de evaluare a dezastrelor naturale pe bază de imagini din dronă

COORDONATOR ȘTIINȚIFIC
Prof. Dr. Ing. Nirvana Popescu

ABSOLVENT,
Mihăilă Nicanor

București
2026

Cuprins

1. Introducere	3
2. Date și preprocesare.....	4
3. Metodologie și implementare.....	7
3.1. Endpoint-uri	8
3.2. Diagrame	9

1. Introducere

Dezastrele naturale produc efecte semnificative asupra comunităților și infrastructurii. Analiza imaginilor poate susține evaluarea rapidă a situației, prioritizarea intervențiilor și comunicarea în timp real. Având la dispoziție colecții de imagini etichetate pentru diverse tipuri de dezastre, precum și dataset-uri pentru multiple tipuri de dezastre, voi proiecta un sistem capabil să parcurgă întregul flux: preluarea imaginilor de la utilizator, preprocesare, analizarea acestora cu ajutorul modelelor / rețelelor neuronale convoluționale, agregarea și stocarea rezultatelor și prezentarea lor într-o interfață prietenoasă.

Acest raport prezintă dezvoltarea unei aplicații complete pentru detectarea și analiza dezastrelor naturale din imagini statice, care integrează trei funcționalități principale: (1) clasificarea tipului de dezastru (de exemplu, inundație, incendiu de vegetație, ciclon, cutremur, accident rutier), (2) detecția obiectelor relevante din imagini pentru estimarea impactului (de exemplu, vehicule, persoane, clădiri și vegetație) și (3) segmentarea semantică a zonelor afectate folosind indicatori cantitativi. Aplicația are o arhitectură care include un backend FastAPI, modele de învățare automată antrenate (ResNet18 pentru clasificare, YOLOv8/ YOLOv8n-oiv pentru detecție și segmentare), un frontend web care permite interacțiunea cu utilizatorul și o bază de date pentru persistența informațiilor relevante.

Conform rapoartelor de antrenare generate de YOLOv8, rezultatele au demonstrat un comportament stabil în clasificare, timpi de procesare adecvați pentru utilizare interactivă și metrice solide în sarcina de segmentare a inundațiilor (mAP + recall). Pentru o analiză practică, sistemul oferă atât feedback calitativ (etichetarea imaginilor) cât și estimări cantitative (procentul de obiecte acoperite în imagine, estimarea suprafeței afectate și numărul de entități identificate).

Scopul proiectului este de a construi o aplicație modulară, extensibilă și ușor de utilizat, care să combine:

- clasificarea în mare a scenei fotografiate (identificarea tipului de dezastru);
- detecția obiectelor și estimări simple de acoperire / impact (suprafața afectată, recomandări cu privire la numărul și tipul de personal necesar intervenției);
- segmentare semantică specifică;
- posibilitatea operatorilor aplicației de a oferi feedback în cazul anumitor erori de clasificare sau recomandare oferite de sistem.

Obiectivele tehnice majore sunt:

- crearea unui model de clasificare pe baza arhitecturii ResNet18, antrenat pe dataset-ul „Natural Disasters” în primă fază, pentru etichetarea imaginilor conform tipului de dezastru din conținutul acestora (Flood, Wildfire, Cyclone, Earthquake, Car Crash);
- utilizarea unui model YOLOv8 pre-antrenat pentru detecție generală de obiecte, adaptat la Open Images V7 (OIV7), pentru redarea estimărilor de impact (număr de persoane/vehicule, dimensiuni aproximative pentru suprafața afectată în funcție de obiectele prezente);
- pregătirea și antrenarea unui model YOLOv8n-seg pentru segmentarea semantică a zonelor de inundație în primă fază de dezvoltare pe baza dataset-ului „Flood Amateur Video for Semantic Segmentation Dataset”, inclusiv conversia măștilor color în formatul YOLO segmentation;
- realizarea unui backend REST robust, bazat pe FastAPI, care expune endpoint-uri de încărcare, predicție, analiză și raportare;
- expunerea unui API pentru încărcarea automată a imaginilor capturate de drone, cu autentificare pe token, endpoint-uri de upload multipart, meta-date (timp, coordonate GPS, altitudine) și confirmare prin webhook;
- proiectarea și implementarea unei baze de date pentru stocarea imaginilor, metadatelor și rezultatelor (clasificare, detecție, segmentare), împreună cu fazele de rulare și configurări, pe o schemă clară cu indexări adecvate;
- containerizarea aplicației în module Docker (backend, frontend, modele), orchestrate cu Docker Compose pentru hosting, configurare prin variabile de mediu, volume persistente și rețele interne;
- proiectarea unui frontend care structurează experiența utilizatorului: încărcare imagine, vizualizare rezultate, detalii analitice.

2. Date și preprocesare

Pentru dezvoltarea sistemului au fost agregate seturi de date provenite din mai multe platforme și surse publice (Kaggle, Roboflow, Open Images V7, arhive academice), cu grade diferite de structurare și calitate. Unele colecții au venit deja organizate pe fișiere standard (train/valid/test) și formate consacrate (COCO, YOLO), cu etichete curate pe clase, în timp ce altele au fost neuniforme: imagini fără adnotări, adnotări parțiale sau inconsistente, rezoluții variabile și distribuții dezechilibrate între clase. În proiect, criteriile de selecție au vizat relevanța pentru fenomenele țintă (inundații, incendii și dezastru naturale în general), acoperirea diversă a scenelor și licențe care permit utilizarea în scop de cercetare. Preprocesarea a inclus standardizarea rezoluției, normalizarea canalelor conform statisticilor ImageNet, deduplicarea și filtrarea imaginilor cu etichete ambigue, precum și conversii de format (ex. transformarea măștilor color în contururi/poligoane YOLO pentru segmentare).

Setul de date „Natural Disasters” (<https://universe.roboflow.com/yolo-classification/natural-disasters>) este organizat în directoarele *train*, *valid* și *test* cu clase separate (Flood, Wildfire, Cyclone, Earthquake, Car Crash). Acesta este folosit pentru antrenarea modelului de clasificare (ResNet18) în (model/train.py) și pentru testare. Imaginile sunt standardizate prin redimensionare la (224, 224) și normalizare pe canale conform statisticilor ImageNet. Am ales să utilizez acest set de date pentru prima etapă prin care trece o imagine sau o secvență de imagini care sunt introduse în aplicație și anume încadrarea acestora într-o clasă din cele prezentate mai sus. De asemenea, numărul mare de imagini disponibile în acest set (aproximativ 9600) asigură o antrenare eficientă și prin urmare un model stabil, cu acuratețe ridicată.

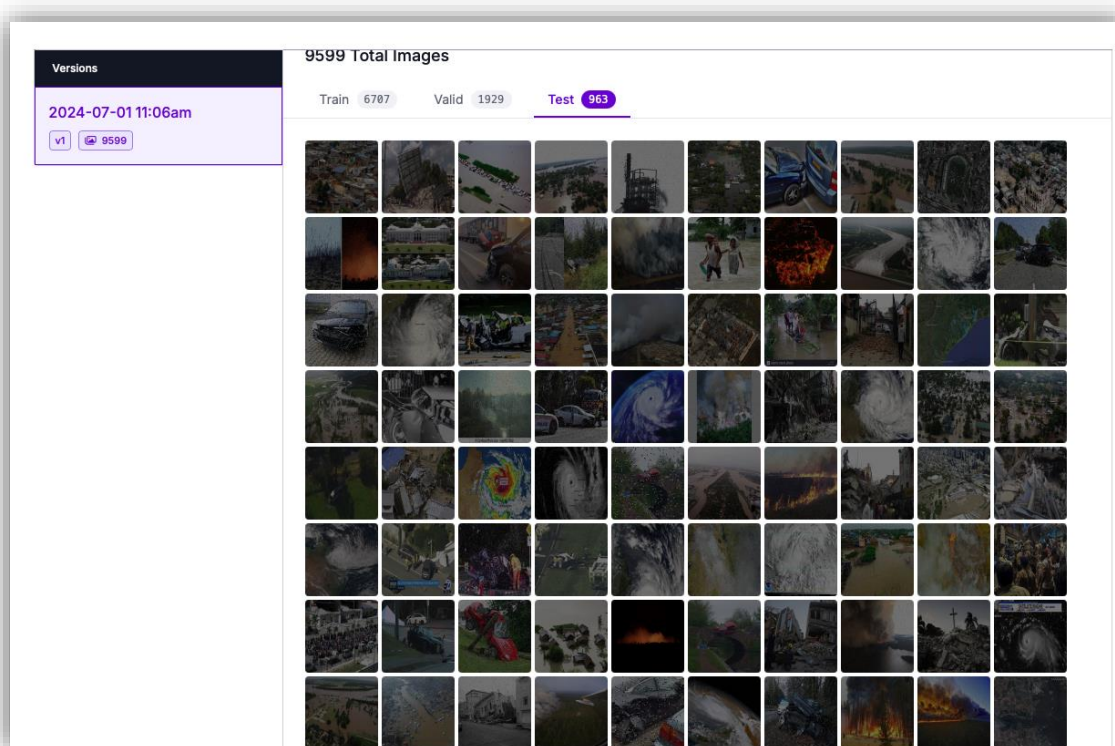


Figura 2.1. Setul de date Natural Disasters

Pentru îmbunătățirea modelului de clasificare, am folosit un set suplimentar de imagini în *fine_tune_images/* cu aceeași structură de clase ca în modelul inițial. Scriptul *fine_tune.py* aplică fine-tuning pe rețeaua ResNet18, cu un *learning rate* crescut (0.0001) și un număr de epoci extins (10), păstrând transformările standard.

Pentru segmentarea semantică a inundațiilor, am descărcat dataset-ul Flood Amateur Video for Semantic Segmentation Dataset (<https://data.mendeley.com/datasets/3kzr8mt8s2/5>) care conține imagini și adnotări sub formă de măști color pentru fiecare cadru. Fișierul (model/prepare_flood_dataset.py) convertește automat măștile RGB în etichete de clasă, extrage poligoanele pentru fiecare

clasă și generează fișierele *labels/*.txt* în format YOLO segmentation, concomitent cu copierea imaginilor în *images/train/* în noul dataset *flood_yolo*. Clasele utilizate în segmentarea de inundații (maparea culori → clase) sunt: 'flood', 'building', 'plant', 'person', 'vehicle', 'sky'. După conversie, se produce fișierul *data.yaml* cu configurația pentru antrenarea YOLOv8n-seg.

Acest set de date este alcătuit din trei directoare: date video brute colectate de pe Instagram, date imagine obținute prin împărțirea videoclipurilor în cadre, și date de adnotare care conțin imagini etichetate în culori în funcție de obiecte. Există 6 clase de obiecte definite prin etichete de culoare, după cum se poate observa și în figura 1: inundații (albastru deschis), clădiri (roșu), plante (verde), persoane (verde salvie), vehicule (portocaliu) și cer (albastru închis). Setul conține date în formate imagine (JPEG/PNG) și video (MP4).



Figura 2.2. Exemplu de segmentare semantică pentru analiza inundațiilor

Transformările folosite consistent în proiect sunt:

- redimensionare la 224x224: uniformizează dimensiunea imaginilor la intrarea modelului (compatibil cu ResNet), prin resampling bilinear, pentru a permite procesarea în loturi și comparabilitatea între eșantioane;

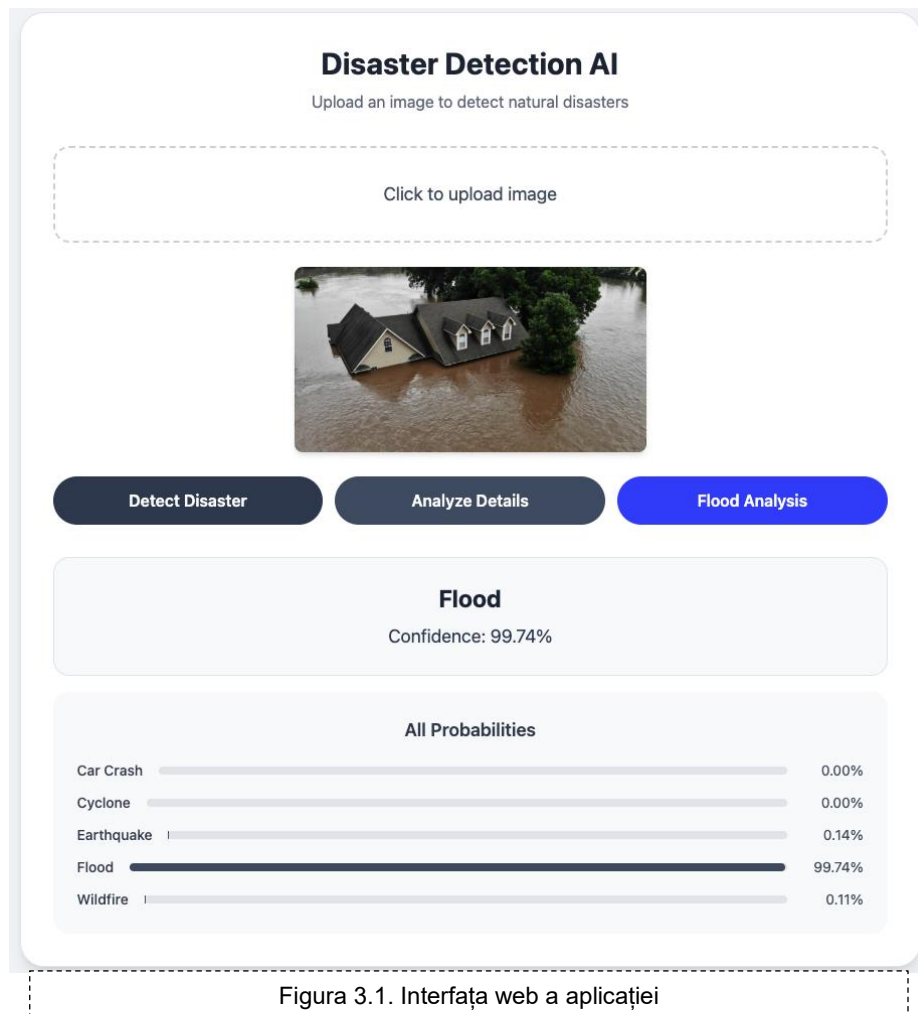
- conversie la tensor: transformă imaginea din format PIL/NumPy în tensor Torch (C×H×W) cu valori în [0,1], necesar pentru operațiile pe GPU/MPS și straturile convoluționale;
- normalizare cu medie [0.485, 0.456, 0.406] și deviație standard [0.229, 0.224, 0.225]: pentru fiecare canal RGB se scade media ImageNet și se împarte la deviația standard; astfel se aliniază distribuția intrărilor cu cea a pre-antrenării, stabilizând optimizarea și predicțiile.

Aceste seturi de date reprezintă doar un punct de plecare pentru antrenarea modelelor pe care le voi folosi în această aplicație. Pe parcurs se vor adăuga mai multe, pentru a spori robustețea modelelor și pentru a crește acuratețea acestora. Ele vor fi documentate pe parcurs și se vor regăsi în lucrarea finală aferentă acestui proiect.

3. Metodologie și implementare

După cum am precizat și anterior, aplicația va cuprinde mai multe module, fiecare cu funcționalitate diferită. Unul dintre acestea este cel responsabil de clasificarea imaginilor în funcție de tipul de dezastru. Modelul de clasificare este implementat în *model/train.py* pe baza arhitecturii ResNet18, încărcată cu ponderi pre-antrenate pe ImageNet. Capul de clasificare *model.fc* este înlocuit cu un strat liniar care produce un vector de probabilități pe cele 5 clase ale dataset-ului „Natural Disasters”. Optimizarea se face cu Adam (`lr=0.001`), pierderea este *CrossEntropyLoss*, iar antrenarea rulează pe 5 epoci cu loturi (batch size) de 32 imagini. Selectarea câștigătorului (best model) se face pe baza acurateții de validare, iar checkpoint-ul salvat conține starea modelului, lista claselor și acuratețea (*disaster_model.pth*).

Am construit logica de clasificare în două endpoint-uri pentru început, */predict* și */file_api*, ambele aparținând fișierului *app.py*, care conține partea de backend și logică. Astfel, în cazul unui request de tip POST realizat din interfața utilizator, imaginea este preprocesată și trecută prin ResNet18, iar sistemul returnează clasa cu probabilitatea maximă și probabilitățile pentru toate clasele, după cum se poate observa în figura 2, care conține o parte din frontend.



3.1. Endpoint-uri

Backend-ul aplicației expune pentru moment trei endpoint-uri REST specializate, fiecare utilizând un model de deep learning diferit pentru sarcini specifice de analiză vizuală. Endpoint-ul */predict* clasifică imaginile folosind ResNet18, */analyze* detectează și numără obiectele prin YOLOv8 antrenat pe Open Images V7, iar */flood_analysis* segmentează zonele inundate cu YOLOv8n-seg. Toate endpoint-urile primesc imagini prin POST requests și returnează rezultate structurate în format JSON, oferind astfel o arhitectură modulară și scalabilă pentru diverse scenarii de analiză.

Endpoint-ul */predict* realizează clasificarea imaginilor folosind modelul ResNet18 pre-antrenat, returnând clasa prezisă împreună cu probabilitățile asociate fiecărei categorii. La primirea unei imagini prin request-ul POST, aceasta este citită din memoria buffer-ului, convertită în format RGB și preprocesată prin transformările standard (redimensionare la 224x224 pixeli, normalizare cu media și deviația standard ImageNet). Input-ul transformat este trecut prin rețeaua neuronală în modul de inferență

(`torch.no_grad()` pentru eficiență), iar output-ul brut (logits) este transformat în probabilități prin funcția softmax. Răspunsul JSON conține trei componente esențiale: clasa prezisă (categoria cu probabilitatea maximă), confidence score-ul (probabilitatea clasei câștigătoare) și un dicționar complet de probabilități pentru toate clasele disponibile în model, permițând astfel analiza gradului de certitudine și a alternativelor posibile pentru fiecare predicție.

Endpoint-ul `/analyze` utilizează modelul YOLOv8 antrenat pe datasetul Open Images V7 pentru detectarea și numărarea obiectelor din imagini, calculând atât numărul de entități, cât și suprafața aproximativă pe care o acoperă. La introducerea imaginii, aceasta este procesată prin rețeaua YOLO care returnează bounding boxes pentru fiecare obiect detectat, împreună cu ID-ul clasei și scorul de încredere. Sistemul contorizează separat vehiculele (mașini, autobuze, camioane) și persoanele, apoi calculează două metrici esențiale de densitate: procentajul de acoperire (raportul dintre aria totală a bounding boxes-urilor și aria imaginii) și aria estimată în metri pătrați folosind o hartă predefinită de dimensiuni medii pentru fiecare clasă de obiect (de exemplu, 0.7m^2 pentru persoane, 12m^2 pentru mașini, 100m^2 pentru clădiri). Răspunsul JSON include numărul de vehicule și persoane, metricile de acoperire spațială, numărul total de obiecte detectate și un dicționar cu toate clasele identificate, oferind o analiză amănunțită a conținutului.

Endpoint-ul `/flood_analysis` utilizează modelul YOLOv8n-seg specializat pentru segmentarea scenelor de inundații. Spre deosebire de detecția clasică cu bounding boxes, modelul de segmentare generează măști binare pentru fiecare obiect detectat, permițând calculul precis al suprafeței inundate prin însumarea pixelilor din masca clasei "flood" raportată la aria totală a imaginii. Sistemul procesează simultan toate măștile detectate pentru a număra obiectele critice din scenă (clădiri, vehicule, persoane, plante) și calculează două metrici esențiale: procentajul de acoperire al inundației (raportul exact dintre pixelii inundați și totalul imaginii) și aria estimată în metri pătrați folosind o conversie aproximativă de $1\% \approx 100\text{m}^2$. Răspunsul JSON conține metrici detaliate despre extinderea inundației, specificarea tuturor categoriilor de obiecte afectate și numărul total de entități identificate.

3.2. Diagrame

Diagrama de activități din figura 3.2.1 prezintă fluxul complet al aplicației de analiză a imaginilor captate de drone, transmise automat către server sau încărcate de utilizatori prin interfața web. Sistemul este structurat pe trei componente principale: Preluarea imaginilor din drone, Frontend (React) și Backend (FastAPI). Dronele captează media (foto/video) și le transmit prin endpoint-ul `/drone/upload` împreună cu metadate esențiale (coordonate GPS, altitudine, timestamp) – acest pas va fi simulat folosind un endpoint

unde vom încărca manual seturi de imagini. Paralel, utilizatorii pot încărca imagini direct prin interfața React, unde pot previzualiza conținutul înainte de a-l trimite spre procesare prin diverse endpoint-uri specializate.

Backend-ul FastAPI orchestrează întregul proces de analiză prin validarea cererilor și rutarea către modelele de machine learning corespunzătoare. În funcție de endpoint-ul apelat, sistemul activează diferite pipeline-uri: ResNet18 pentru clasificarea imaginilor (endpoint /predict și /file_api), YOLOv8 Detection pentru detectarea obiectelor și calculul densității (endpoint /analyze), sau YOLOv8 Segmentation pentru analiza inundațiilor și segmentarea zonelor afectate (endpoint /flood_analysis). Toate rezultatele - imagini procesate, predicții, metadata și log-uri - sunt stocate în baza de date, iar răspunsurile sunt returnate în format JSON structurat, asigurând trasabilitatea completă a procesului de analiză.

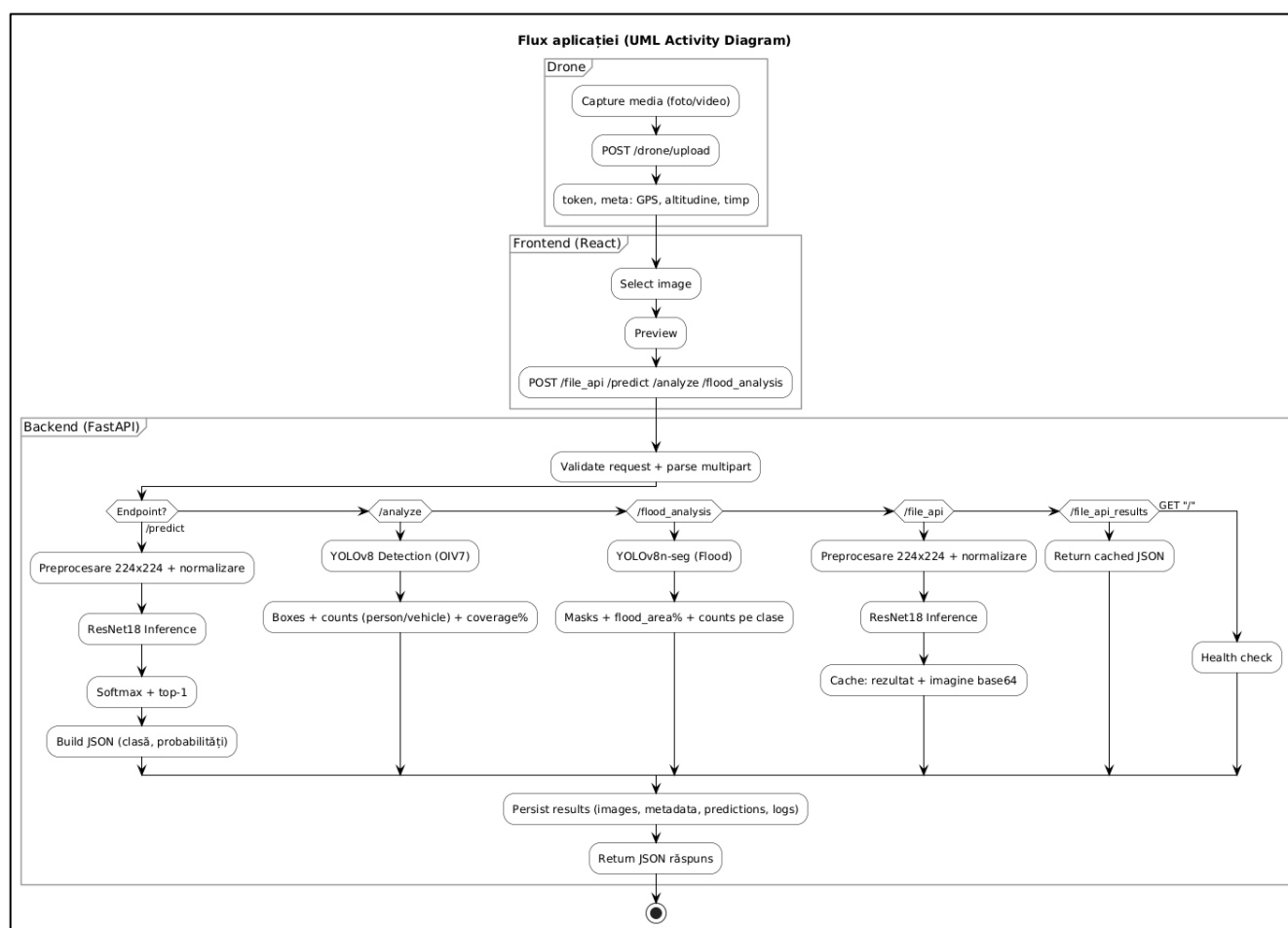


Figura 3.2.1. Arhitectura de procesare pe backend

Diagrama din Figura 3.2.2 prezintă fluxul sistemului: dronele trimit automat imagini către backend pentru procesare, iar utilizatorii primesc în UI doar alertele cu dezastre detectate, însoțite de metadata detaliate (tip, scor încredere, suprafață, recomandări).

Sistemul include un mecanism de feedback loop prin care imaginile marcate ca neconforme sunt salvate automat în foldere */retraining/{tip_dezastru}/* pentru fine-tuning. Paralel, există o interfață de testare manuală care procesează imaginile încărcate de utilizatori prin același pipeline de analiză, asigurând consistența rezultatelor.

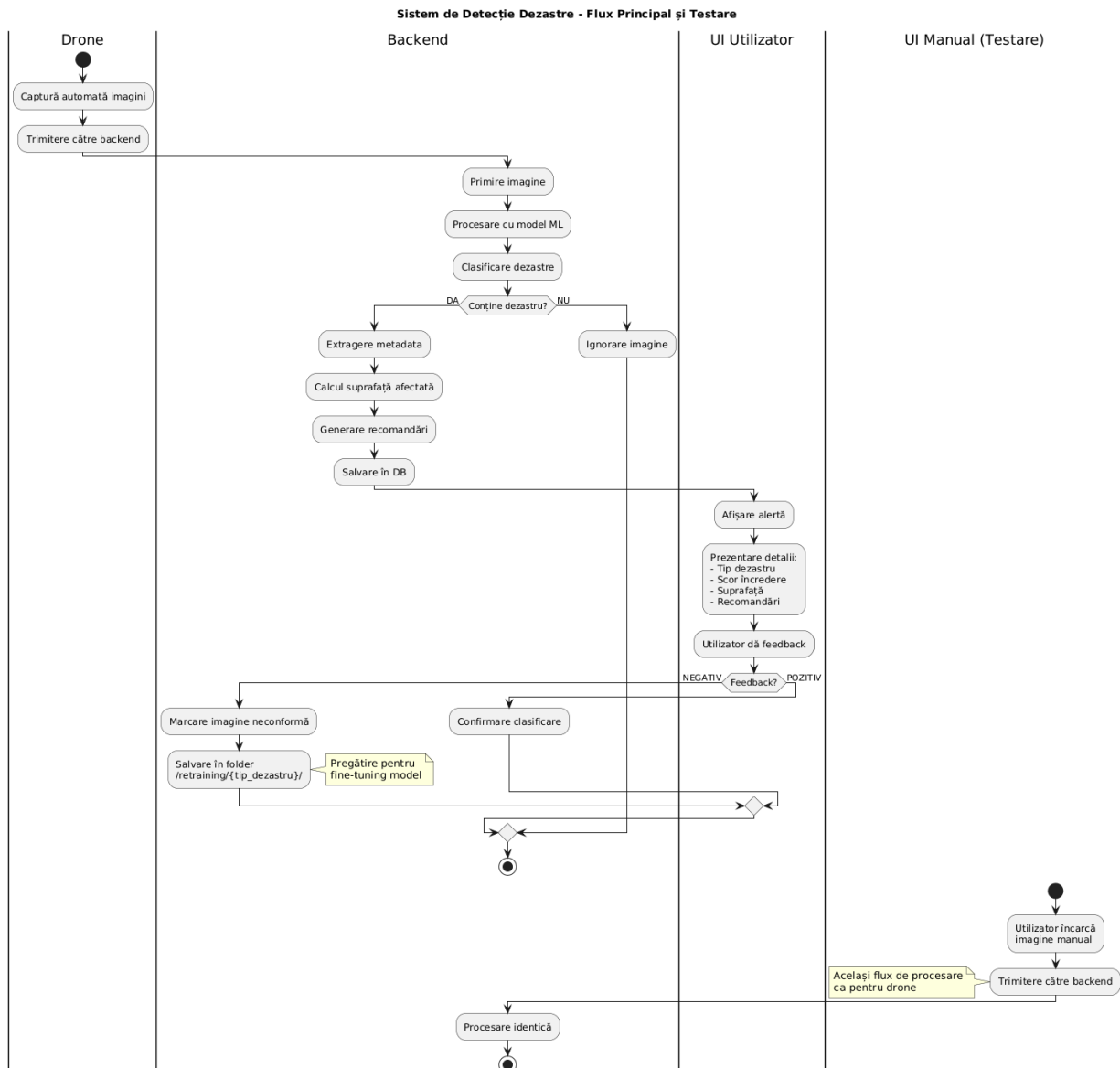


Figura 3.2.2. Fluxul de detecție și feedback